Sprint 1 - Consolidación idea de proyecto: Fast Order Cúcuta - Sistema de Gestión Integrada de Pedidos e Inventario.

Carrera: Ingeniería de Sistemas Asignatura:

Asignatura: Proyecto final de ingeniería de sistemas

Docente: Jover Alonso Cabrales Pineda

Elaborado por:

Josue David Ojeda Vivas

Johan Humberto Villegas

Javier alejandro serrano Ruiz

Kevin Eduardo Garcia Moreno

Introducción

Este documento presenta el diseño de Fast Order Cúcuta. Es una idea para ayudar a los negocios pequeños de comidas rápidas a organizar sus pedidos. Primero se propone un producto mínimo viable (MVP), que es una versión inicial con las funciones más importantes para probar si la solución funciona en un negocio real.

1. Identificación y resumen

- **Nombre del Proyecto:** Fast Order Cúcuta - Sistema de Gestión Integrada de Pedidos e Inventario.
- **Sector de Aplicación:** Gastronomía / Comida rápida (Micronegocios en el área metropolitana de Cúcuta).

Fast Order Cúcuta es una plataforma web y móvil liviana para micronegocios de comida rápida. Junta en un solo flujo los pedidos que llegan por WhatsApp y los que se toman en el mostrador, y lleva el inventario en tiempo real.

Hoy estos negocios funcionan a pulso: cuaderno de papel y cuentas personales de WhatsApp. El resultado es una cocina que se atasca, cambios que se pierden en el camino (un “sin cebolla” que nadie anotó, una adición que nunca se cobró) e insumos perecederos que se dañan antes de usarse.

La plataforma cambia el cuaderno por una pantalla para tomar pedidos rápido. Lo que se digita ahí aparece de inmediato en la cocina, en un tablero Kanban (Kitchen Display System, o KDS), y una calculadora descuenta del inventario los insumos de cada plato.

2. Problema y Contexto (Estado del Arte Breve)

- ## Contexto Operativo

En muchos negocios de comidas rápidas de Cúcuta, los pedidos llegan por dos medios: en el local o por WhatsApp. A veces el dueño recibe los pedidos, cobra, ayuda en la cocina y empaca al mismo tiempo. Cuando hay muchos clientes, especialmente en la noche y los fines de semana, se puede perder una orden o entregar algo incorrecto.

También se usan notas de papel y mensajes que se van quedando arriba en el chat. El papel puede mancharse o ser difícil de leer. Además, se pueden olvidar indicaciones como “sin cebolla” o “salsa aparte”. Esto puede causar quejas, desperdicio de comida y pérdida de dinero. Como los ingredientes no se revisan todos los días, el negocio puede quedarse sin algo importante o comprmbién participan en la entrega, pero no usarán la primera versión del sistema.

● Problema operativo

El problema central radica en la desorganización en la consolidación de pedidos multicanal y el cálculo empírico de insumos diarios. El fenómeno se modela como un sistema de colas con múltiples fuentes de entrada no sincronizadas:

- **Degradación en la cola de atención:** En las horas de mayor movimiento, entran una mayor cantidad de pedidos,sumando los de WhatsApp y los del mostrador, lo cual puede saturar la tasa de procesamiento del propietario, En WhatsApp, los mensajes nuevos desplazan hacia abajo a los antiguos sin clasificación de estado, mientras que en cocina los pedidos en papel se puede manchar de grasa y puede ocurrir un error en la caligrafía

- **Pérdida financiera y merma:** La omisión de adiciones o instrucciones especiales genera reprocesos, quejas por pedidos fríos o incompletos, e incapacidad de distinguir entre el flujo de caja diario y la utilidad neta real del negocio.

● Estado del arte breve

Alternativa actual	Qué hace	Deficiencias identificadas
Papel / libreta	Anota pedidos a mano	La letra rápida es ilegible, el papel se mancha de grasa, se pierde y no permite métricas de ventas
WhatsApp personal / Business básico	Recibe pedidos por chat	Los mensajes nuevos desplazan a los antiguos, no hay estados del pedido, se olvidan adiciones que luego no se cobran
Sistemas POS genéricos	Registran ventas en caja	Pensados para restaurantes formales, más costosos y complejos, sin integración natural con pedidos por chat ni con la cocina de un micronegocio
Plataformas de domicilios de terceros	Intermedian pedidos y entregas	Comisiones altas sobre márgenes ajustados; no resuelven la gestión interna de cocina ni inventario

3. Objetivos y alcance

● Objetivo General:

Desarrollar e implementar, una plataforma centralizada de gestión de pedidos e inventario que reduzca en al menos un **40% los errores operativos** y mejore la rentabilidad de los micronegocios de comida rápida en Cúcuta

● Objetivos Específicos:

- Diagnosticar el proceso actual de toma de pedidos y control de inventario en los negocios piloto, estableciendo la línea base de errores operativos, desperdicio de insumos y tiempos de despacho.

- Desarrollar un módulo de pedidos que reúna en una sola pantalla de cocina las órdenes de WhatsApp y las del mostrador, con botones predefinidos para modificar los platos.

- Desarrollar un módulo de inventario que automatice el registro de consumo diario de ingredientes clave y genere alertas de stock bajo.

- Implementar la plataforma en los negocios piloto, garantizando que al menos el 95% de las órdenes queden registradas digitalmente.

- Validar el desempeño de la plataforma frente a la línea base, comprobando una reducción de al menos un 40% en errores operativos, un 25% en desperdicio de perecederos y un 30% en el tiempo de despacho en horas pico.

● Alcance del Proyecto:

1. **dentro del alcance:**

- Toma rápida de pedidos en caja (tablet o celular) con botones predefinidos de adiciones y modificaciones

- Ingreso de pedidos de WhatsApp al mismo flujo (vía WhatsApp Business API).

- Tablero Kanban de cocina y entregas con tres estados.

- Módulo básico de inventario: descuento automático por plato y alertas de stock bajo.

- Modo offline temporal con sincronización posterior.

2. **Fuera del alcance**

- Contabilidad completa, facturación electrónica y nómina.
- Rastreo GPS de motorizados independientes.
- Pasarela de pagos en línea.
- Reportes avanzados de rentabilidad (se evalúan tras validar pedidos y Kanban).

4. Requisitos

- Requisitos Funcionales:

ID	Actor	Requisito	Prioridad
RF-01	Cajero	El sistema permitirá registrar un pedido presencial en caja con selección de productos mediante botones.	Alta
RF-02	Cajero	El sistema permitirá añadir modificaciones y adiciones predefinidas ("sin cebolla", "salsa aparte") y calcular su cobro.	Alta
RF-03	Cliente final / WhatsApp Business Platform	El sistema recibirá pedidos de WhatsApp Business y los convertirá en tickets digitales.	Alta
RF-04	Cocinero	El sistema mostrará en una pantalla de cocina los tickets en tablero Kanban: pendiente, en preparación, despachado.	Alta
RF-05	Cocinero	El personal podrá cambiar el estado de un pedido con una sola acción táctil.	Alta
RF-06	Cocinero	El sistema mostrará de forma destacada las modificaciones de cada pedido en la pantalla de cocina.	Alta
RF-07	Sistema	El sistema descontará insumos del inventario según la receta de cada plato vendido.	Media
RF-08	Propietario	El sistema alertará cuando un insumo clave llegue a un umbral mínimo configurable.	Media
RF-09	Propietario	El sistema permitirá registrar manualmente entradas de inventario y mermas.	Media
RF-10	Propietario	El sistema generará un resumen diario de ventas y consumo de insumos.	Baja
RF-11	Cajero / Motorizado independiente	El sistema registrará el domicilio y el motorizado asignado a cada entrega.	Baja
RF-12	Propietario	El sistema gestionará usuarios con roles (propietario, cajero, cocina).	Alta
RF-13	Sistema	El sistema registrará marcas de tiempo por cambio de estado para medir tiempos de despacho.	Alta

- Requisitos no funcionales:

ID	Atributo	Requisito	Prioridad	Verificación
RNF-01	Usabilidad	Un cajero nuevo debe poder registrar un pedido sin ayuda tras una capacitación práctica de máximo 30 minutos; botones grandes, iconografía clara y colores de estado.	Alta	Prueba con usuarios reales
RNF-02	Rendimiento	Un pedido registrado debe aparecer en la cocina en máximo 2 segundos con conexión estable.	Alta	Pruebas de latencia
RNF-03	Disponibilidad	Disponibilidad objetivo de 99 % en horario de operación (noches y fines de semana).	Media	Monitoreo
RNF-04	Fiabilidad	Modo offline: caja y cocina siguen operando sin internet y sincronizan sin perder ni duplicar pedidos.	Alta	Pruebas de desconexión
RNF-05	Seguridad	Autenticación por roles, comunicaciones cifradas (HTTPS) y copias de seguridad diarias.	Alta	Revisión de seguridad
RNF-06	Privacidad	Tratamiento de datos de clientes (teléfono, dirección) conforme a la Ley 1581 de 2012 (Habeas Data), con autorización y finalidad definidas.	Alta	Lista de verificación legal
RNF-07	Compatibilidad	Funcionamiento en navegadores móviles y de escritorio recientes y en tablets Android económicas.	Media	Pruebas multidispositivo
RNF-08	Escalabilidad	Soportar al menos 65 negocios (meta del año 3) sin rediseño de la arquitectura.	Baja	Pruebas de carga
RNF-09	Mantenibilidad	Código versionado, documentado y con pruebas automatizadas sobre la lógica núcleo.	Media	Revisión de código
RNF-10	Costo	Suscripción accesible de $40.000 COP mensuales por negocio.	Media	Análisis financiero

5. Arquitectura y diseño

Se compararon tres alternativas de implementación con seis criterios: costo, compatibilidad con los dispositivos del negocio, facilidad de despliegue, funcionamiento sin internet, integración con WhatsApp y qué tan bien se adaptan a un micronegocio de márgenes estrechos.

Luego se justifica la alternativa elegida y se documenta su arquitectura con dos enfoques que se complementan. El modelo C4 se muestra por niveles (contexto, contenedores y componentes).

- Alternativas evaluadas

Criterio	A. Aplicación web progresiva (PWA) con backend en la nube	B. App móvil nativa (Android)	C. Adaptar un POS genérico existente
Costo de desarrollo	Medio	Alto	Bajo inicial, licencias recurrentes
Compatibilidad con tablets y celulares	Alta (navegador)	Solo Android	Variable
Despliegue y actualizaciones	Inmediatos	Requiere tiendas y actualización por dispositivo	Depende del proveedor
Modo offline	Posible (service workers)	Posible	Limitado
Integración con WhatsApp	Flexible	Flexible	Generalmente no disponible
Ajuste a la operación del micronegocio	Alto	Alto	Bajo

Decisión: se selecciona la **alternativa A (PWA + backend en la nube)** por menor costo, una sola base de código para caja y cocina, despliegue inmediato y mejor ajuste al presupuesto y al plazo.

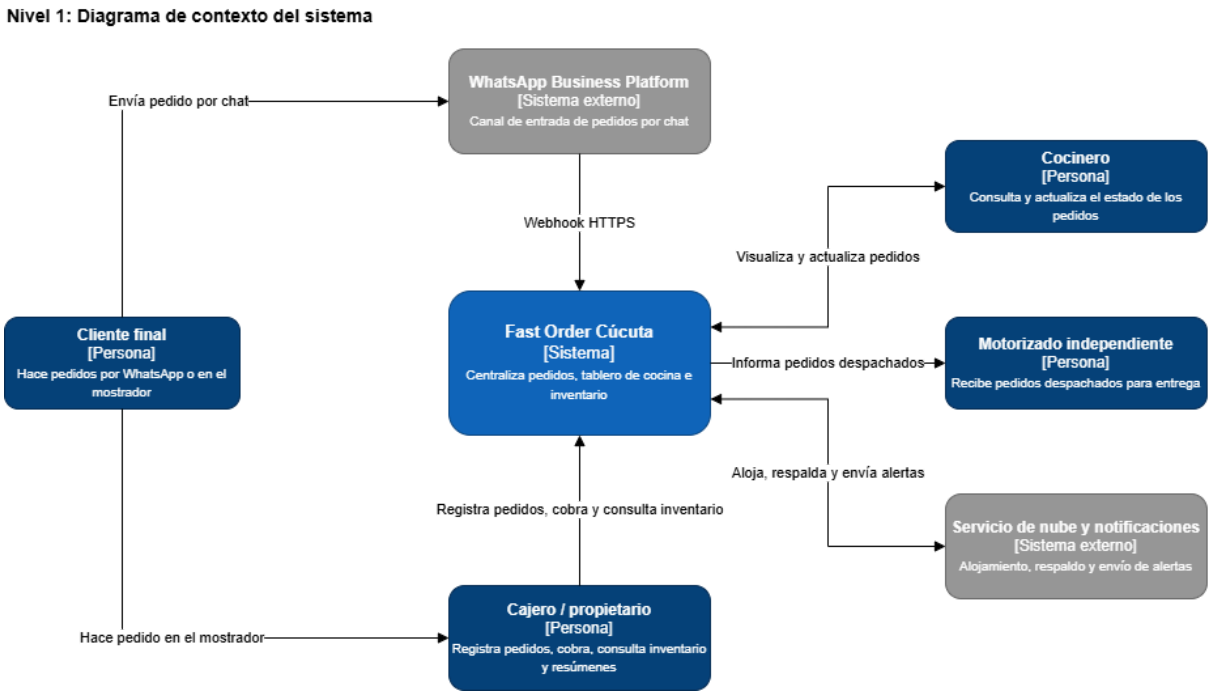
- Vista C4 – Nivel 1: Contexto del sistema

El nivel 1 del modelo C4 presenta Fast Order Cúcuta como una sola pieza y lo ubica en su entorno:

quiénes lo usan y con qué sistemas externos se conecta. Es la vista más general y sirve para que cualquier lector entienda quién participa en el flujo de un pedido, desde el cliente que lo hace hasta el cocinero que lo prepara, sin entrar todavía en detalles técnicos.

Elemento	Tipo	Descripción
Cliente final	Persona	Hace pedidos por WhatsApp o en el mostrador
Cajero / propietario	Persona	Registra pedidos, cobra, consulta inventario y resúmenes
Cocinero	Persona	Consulta y actualiza el estado de los pedidos
Motorizado independiente	Persona	Recibe pedidos despachados para entrega
Fast Order Cúcuta	Sistema	Centraliza pedidos, tablero de cocina e inventario
WhatsApp Business Platform	Sistema externo	Canal de entrada de pedidos por chat
Servicio de nube y notificaciones	Sistema externo	Alojamiento, respaldo y envío de alertas

Nivel 1: Diagrama de Contexto del Sistema



- Vista C4 – Nivel 2: Contenedores

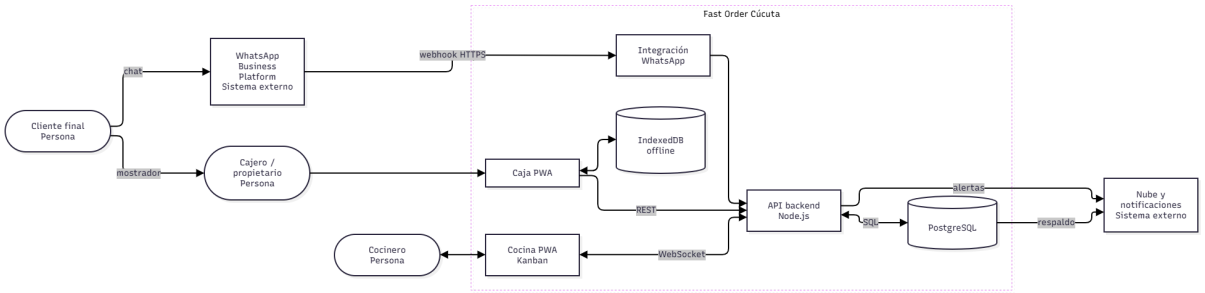
El nivel 2 del modelo C4 abre la caja negra de Fast Order Cúcuta y muestra las piezas de software que lo componen: las aplicaciones que usan el cajero y el cocinero, el servicio que

procesa la lógica del negocio, la integración con WhatsApp y la base de datos. Para cada contenedor se indica la tecnología sugerida y su responsabilidad, de modo que se vea dónde ocurre cada función y cómo se comunican entre sí.

Contenedor	Tecnología sugerida	Responsabilidad
Aplicación de caja (PWA)	HTML/JS, framework SPA, service worker	Toma rápida de pedidos, modo offline
Aplicación de cocina (PWA)	Mismo código base	Tablero Kanban en tiempo real
API backend	Node.js (o equivalente) con API REST y WebSockets	Reglas de negocio, autenticación, sincronización
Servicio de integración WhatsApp	Webhook sobre la API de WhatsApp Business	Recibe mensajes y los transforma en pedidos
Base de datos	PostgreSQL (o MySQL)	Pedidos, productos, recetas, inventario
Almacenamiento local del navegador	IndexedDB	Cola de pedidos pendientes de sincronización en modo offline

SERVICE CLAUDE

Nivel 2: Diagrama de contenedores



● Vista C4 – Nivel 3: Componentes del backend

El nivel 3 del modelo C4 se enfoca en un solo contenedor, la API backend, y muestra los módulos internos que manejan la lógica del negocio: pedidos, tablero Kanban, inventario, integración con mensajería, usuarios, sincronización y reportes. Esta vista sirve de guía para repartir el trabajo entre los desarrolladores y para que cada responsabilidad quede en su propio módulo. Así es más fácil probar el sistema y cambiarlo después.

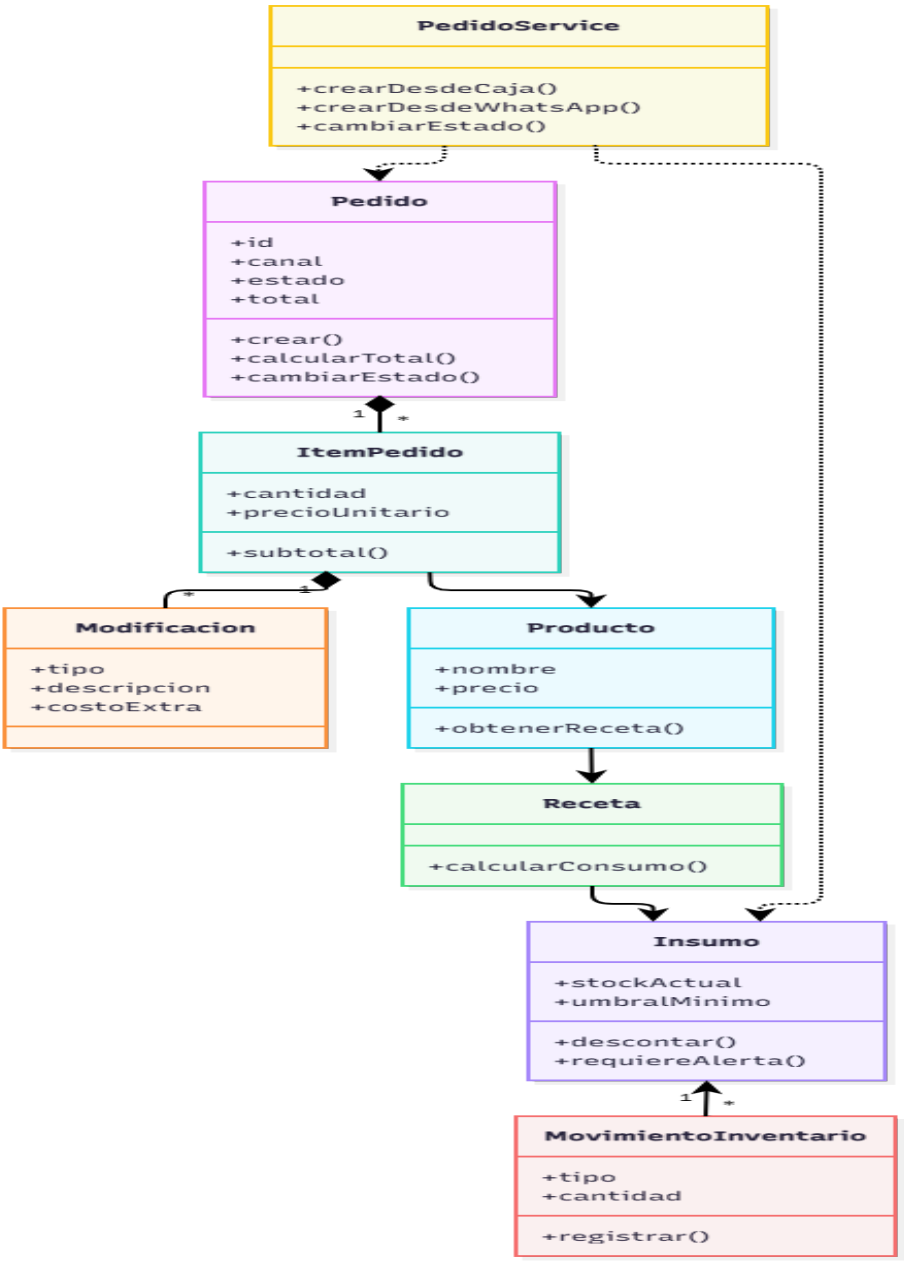
- **Módulo de pedidos:** creación, modificaciones, cálculo de totales.
- **Módulo Kanban:** máquina de estados (pendiente → en preparación → despachado) y emisión de eventos en tiempo real.
- **Módulo de inventario:** recetas, descuento de insumos, umbrales y alertas.
- **Módulo de integración de mensajería:** análisis del mensaje y creación del ticket.
- **Módulo de usuarios y roles:** autenticación y autorización.
- **Módulo de sincronización:** resolución de conflictos y de duplicación tras reconexión.
- **Módulo de reportes:** resumen diario de ventas y consumo.

● Vista C4 – Nivel 4: Código

El nivel 4 del modelo C4 es el más detallado: baja de los módulos al código y muestra las clases principales con sus atributos y responsabilidades. Se aplica a los módulos de mayor valor del sistema, pedidos e inventario, que concentran las reglas de negocio de los objetivos OE1 a OE4.

Clase	Atributos principales	Métodos / responsabilidades
Pedido	id, canal (caja o WhatsApp), estado, fechaIngreso, total, negocioId	crear(), calcularTotal(), cambiarEstado()
ItemPedido	productoId, cantidad, precioUnitario	subtotal()
Modificacion	tipo (sin ingrediente, extra, aparte), descripcion, costoExtra	aplicarA(item)
EstadoPedido (enumeración)	PENDIENTE, EN_PREPARACION, DESPACHADO	Define los tres estados del tablero Kanban
Producto	id, nombre, precio	obtenerReceta()
Receta	productoId, lista de insumos con su cantidad	calcularConsumo(cantidad)
Insumo	id, nombre, unidad, stockActual, umbralMinimo	descontar(cantidad), requiereAlerta()
MovimientoInventario	insumoId, tipo (entrada, consumo o merma), cantidad, fecha	registrar()
PedidoService	repositorio de pedidos, servicio de inventario	crearDesdeCaja(), crearDesdeWhatsApp(), cambiarEstado(pedidoId, nuevoEstado)

Nivel 4: Diagrama del codigo



Vista	Contenido en este proyecto
Lógica	Entidades: Negocio, Usuario, Pedido, ÍtemPedido, Modificación, Producto, Receta, Insumo, MovimientoInventario, Entrega. Un Pedido tiene varios ítems y cada ítem varias modificaciones; un Producto se compone de insumos mediante su Receta.
Procesos	Llega el pedido (caja o WhatsApp) → se valida y se guarda → se emite el ticket a cocina por WebSocket → cocina cambia el estado → al despachar se descuenta inventario → se evalúa el umbral y se alerta. Concurrencia gestionada con transacciones y colas.
Desarrollo	Repositorio Git con módulos de frontend, backend e integración; ramas por funcionalidad; integración continua con pruebas automáticas.
Física (despliegue)	Dispositivos del local (tablet de caja, pantalla de cocina, router) conectados por HTTPS a la nube (AWS, Google Cloud o Azure), donde corren API y base de datos con respaldo diario.
Escenarios (+1)	(1) Pedido presencial con "sin cebolla"; (2) pedido por WhatsApp en hora pico; (3) caída de internet y recuperación; (4) alerta de insumo bajo.

Decisiones de diseño relevantes

1. **PWA sobre app nativa:** menor costo y despliegue inmediato.
2. **Base de datos relacional:** integridad transaccional para pedidos e inventario.
3. **Eventos en tiempo real (WebSockets):** la cocina ve el pedido sin refrescar.
4. **Offline-first en caja:** los fines de semana son el momento crítico y la conectividad es un riesgo conocido.
5. **Botones predefinidos en lugar de texto libre:** reduce errores y acelera el registro.
6. **Priorización por fases:** pedidos y Kanban primero, inventario después.

6. Plan de implementación

- Fases

Fase	Duración	Descripción	Resultado esperado	Responsable
1. Requisitos y diseño UX/UI	1 semana (sem. 1)	Historias de usuario y wireframes de toma rápida	Prototipo y especificaciones aprobadas	Analista / Diseñador UX
2. Desarrollo del MVP	3 semanas (sem.2–4)	Módulo de pedidos, tablero de cocina e integración WhatsApp	Software funcional listo para pruebas	Desarrolladores
3. Pruebas piloto	1 semana (sem. 5)	Prueba en un negocio local en hora pico	Métricas de errores y lista de ajustes	QA / Líder de proyecto
4. Despliegue y ajustes	2 semanas (sem. 6–7)	Integración de inventario, capacitación e instalación	Plataforma instalada y operativa	Implementador / Soporte

- Distribución de responsabilidades

Integrante	Rol principal
Josue David Ojeda Vivas	Líder de proyecto y analista
Johan Humberto Villegas	Desarrollador frontend
Javier Alejandro Serrano Ruiz	Desarrollador backend
Kevin Eduardo García Moreno	Integración y control de calidad (QA)

- EDT individual: actividades y tareas
- Josue David Ojeda Vivas

Líder de proyecto y analista

Su responsabilidad principal será organizar el desarrollo del proyecto, identificar las necesidades del negocio y verificar que las actividades se cumplan de acuerdo con lo establecido.

Código	Tarea	Descripción
1.1	Levantamiento de requisitos	Identificar las necesidades de los negocios de comidas rápidas y establecer las funcionalidades del sistema.
1.2	Historias de usuario	Definir las funcionalidades desde la perspectiva del propietario, cajero y cocinero.

1.3	Planificación	Organizar las actividades, los tiempos de ejecución y los entregables de cada sprint.
1.4	Seguimiento del proyecto	Supervisar el cumplimiento de las tareas y coordinar el trabajo entre los integrantes.
1.5	Documentación	Elaborar y organizar la documentación técnica y administrativa del proyecto.
1.6	Informe de resultados	Consolidar los resultados obtenidos durante las pruebas piloto.

Entregables: documento de requisitos, historias de usuario, planificación, informes de seguimiento y documentación final.

- Johan Humberto Villegas

Desarrollador frontend

Será responsable de construir la parte visual de Fast Order Cúcuta, procurando que las interfaces sean sencillas, intuitivas y adecuadas para los dispositivos utilizados en los negocios.

Código	**Tarea**	**Descripción**
2.1	Diseño UX/UI	Diseñar la estructura visual y los prototipos de las interfaces del sistema.
2.2	Interfaz de caja	Desarrollar la pantalla para registrar pedidos presenciales mediante botones y selección de productos.
2.3	Modificaciones de pedidos	Implementar las opciones visuales para agregar modificaciones como «sin cebolla» o «salsa aparte».

2.4	Interfaz de cocina	Construir la pantalla donde el personal de cocina pueda consultar los pedidos recibidos.
2.5	Tablero Kanban	Implementar la visualización de los estados pendientes, en preparación y despachado.
2.6	Interfaz offline	Adaptar la aplicación para que pueda utilizarse temporalmente sin conexión.

Entregables: prototipos UX/UI, interfaz de caja, interfaz de cocina y tablero Kanban.

- Javier Alejandro Serrano Ruiz

Desarrollador backend

Se encargará de desarrollar la lógica interna del sistema, gestionar la información y garantizar que los módulos de pedidos e inventario funcionen correctamente.

Código	**Tarea**	**Descripción**
3.1	Diseño de base de datos	Estructurar las tablas y relaciones necesarias para almacenar usuarios, pedidos, productos e inventario.
3.2	Configuración del backend	Preparar el entorno de desarrollo y establecer la conexión entre el servidor y la base de datos.
3.3	Gestión de pedidos	Programar las funciones para crear, almacenar, consultar y administrar los pedidos.
3.4	Cálculo de pedidos	Desarrollar la lógica para calcular precios, cantidades, adiciones y modificaciones de productos.

3.5	Desarrollo de API	Crear los servicios que permitirán la comunicación entre el frontend y el backend.
3.6	Módulo de inventario	Implementar el descuento automático de insumos según las recetas de cada producto.
3.7	Alertas de inventario	Programar las alertas cuando los ingredientes alcancen el nivel mínimo establecido.

Entregables: base de datos, API backend, lógica de gestión de pedidos y módulo de inventario.

- Kevin Eduardo García Moreno

Integración y control de calidad (QA)

Será responsable de conectar los diferentes componentes del sistema, verificar su funcionamiento y realizar las pruebas necesarias antes de su implementación.

Código	**Tarea**	**Descripción**
4.1	Configuración técnica	Apoyar la preparación del entorno de trabajo y la organización del repositorio Git.
4.2	Integración con WhatsApp	Implementar la comunicación con WhatsApp Business API mediante webhooks para convertir los mensajes recibidos en pedidos digitales.
4.3	Sincronización offline	Verificar que los pedidos registrados sin conexión se sincronicen correctamente al recuperar internet.
4.4	Pruebas unitarias	Comprobar que las funciones individuales del sistema trabajen correctamente.

4.5	Pruebas de integración	Verificar la comunicación entre frontend, backend, base de datos y WhatsApp.
4.6	Pruebas funcionales	Comprobar el recorrido completo de los pedidos, desde su registro hasta su despacho.
4.7	Pruebas de rendimiento	Evaluar los tiempos de respuesta y el comportamiento del sistema durante horas de alta demanda.
4.8	Apoyo al despliegue	Colaborar con la instalación, configuración y validación de la plataforma en el negocio piloto.

Entregables: integración con WhatsApp, informes de pruebas, resultados de validación y apoyo en el despliegue.

- Cronograma de implementación

Actividad	Sem. 1	Sem. 2–4	Sem. 5	Sem. 6–7
Requisitos y diseño UX/UI	•			
Desarrollo del MVP		•		
Pruebas piloto			•	
Despliegue y ajustes				•

- Entregables de implementación

Hito	Semana	Entregable
H1	2	Especificación de requisitos y prototipo aprobado
H2	4	Módulo de pedidos y toma rápida funcional
H3	5	MVP con tablero Kanban e integración WhatsApp
H4	6	Informe del piloto con métricas y lista de ajustes
H5	8	Plataforma operativa con inventario, capacitación y cierre

- Propuesta de planificación por Sprints

Sprint	Duración	Objetivo del Sprint (Sprint Goal)	Requisitos / Tareas asociadas (Backlog)	Entregable / Hito
Sprint 1: Diseño, Arquitectura y Base de Datos	Semanas 1–2	Definir la experiencia de usuario (UX/UI), configurar el entorno en la nube y estructurar la base de datos y autenticación.	• Levantamiento de línea base en negocios piloto. • Diseño de wireframes de toma rápida y Kanban. • Esquema en PostgreSQL y configuración de repositorio Git. • RF-12: Gestión de usuarios y roles (propietario, cajero, cocina). • RNF-01, RNF-05: Usabilidad y seguridad (HTTPS/roles).	H1 (Sem. 2): Especificación de requisitos, arquitectura base y prototipo UX/UI aprobado.
Sprint 2: Módulo de Caja y Toma Rápida (Core MVP)	Semanas 3–4	Construir el flujo presencial de pedidos en mostrador con soporte para modificaciones y modo offline.	• RF-01: Registro de pedido presencial mediante botones. • RF-02: Adiciones y modificaciones predefinidas ("sin cebolla", "salsa aparte"). • RNF-04: Modo offline temporal con IndexedDB y Service Workers. • RF-11: Registro básico de domicilio y motorizado.	H2 (Sem. 4): Módulo de pedidos en caja y toma rápida funcional (online/offline).
Sprint 3: Tablero Kanban (KDS) y WhatsApp	Semana 5	Sincronizar la caja con la pantalla de cocina en tiempo real e integrar la recepción de pedidos por WhatsApp.	• RF-03: Webhook de WhatsApp Business API para crear tickets digitales. • RF-04, RF-05, RF-06: Tablero Kanban en tiempo real (WebSockets) con cambio de estado táctil y alertas visuales de modificaciones. • RF-13, RNF-02: Marcas de tiempo por estado y latencia <2 s< 2\text{ s}<2 s.	H3 (Sem. 5): MVP completo con tablero Kanban en tiempo real e integración con WhatsApp.
Sprint 4: Piloto, Inventario y Despliegue Final	Semanas 6–8	Validar el MVP en operación real (hora pico) e incorporar el control de inventario, alertas y reportes.	• Semana 6 (Piloto): Pruebas en local en hora pico, medición de tiempos y corrección de errores. • RF-07, RF-08, RF-09: Descuento automático por receta, alertas de stock mínimo y registro de entradas/mermas. • RF-10: Resumen diario de ventas y consumo. • Capacitación de 30 min al personal y puesta en producción.	H4 (Sem. 6): Informe del piloto con métricas. H5 (Sem. 8): Plataforma operativa con inventario, capacitación y cierre.

7. Riesgos e impactos

- Matriz de riesgos

ID	Riesgo	Prob.	Impacto	Prioridad	Mitigación
R1	Resistencia al cambio del personal de cocina	Alta	Alto	Crítica	Interfaz de curva de aprendizaje mínima (iconos claros, botones grandes, colores de estado) y capacitación práctica previa al lanzamiento
R2	Fallas de conectividad en horario pico de fin de semana	Media	Alto	Alta	Modo offline con sincronización posterior; recomendar router estable
R3	Baja adopción por el costo de la suscripción	Media	Alto	Alta	Prueba gratuita de 30 días durante el piloto y ajuste del precio según disposición real de pago
R4	Retrasos en el MVP por ser el primer proyecto del equipo en este segmento	Media	Alto	Alta	Priorizar funciones núcleo (pedidos y Kanban) sobre inventario y recortar el alcance si el cronograma se retrasa
R5	Pérdida o daño de hardware por grasa, calor o golpes	Media	Bajo	Media	Fundas protectoras, protocolo de limpieza y dispositivo de reemplazo
R6	Dependencia de la API de WhatsApp Business (aprobación, costos por conversión, cambios de política)	Media	Alto	Alta	Ingreso manual de pedidos de chat como alternativa y diseño desacoplado del canal
R7	Incumplimiento de protección de datos personales de clientes	Baja	Alto	Media	Aviso de privacidad, autorización, mínimos datos necesarios y control de acceso por roles
R8	Estimación de mercado (1.200 negocios) sin respaldo formal	Media	Medio	Media	Validar con encuestas y visitas durante el piloto y ajustar proyecciones

Impactos esperados

- Operativo: menos errores, menos tiempos de espera y una cocina que recibe cada pedido con la información completa.
- Económico: se pierde menos dinero por adiciones que nadie cobró y por desperdicio, y el dueño puede ver cuánto gana realmente cada día.
- Social: el emprendedor se desgasta y se estresa menos, y los micro negocios pueden formalizarse de forma progresiva.
- Ambiental: se gasta menos papel y se desperdician menos alimentos.
- Impactos negativos a vigilar: la brecha digital para quienes saben poco de tecnología, la dependencia de un proveedor tecnológico y el costo del hardware para negocios con márgenes ajustados.

8. Plan de pruebas y validación

Se aplica una estrategia por niveles, alineada con ISO/IEC/IEEE 29119 y con los criterios de calidad de ISO/IEC 25010.

- Plan de estrategia

Nivel	Qué válida	Herramientas sugeridas	Responsable
Unitarias	Cálculo de totales, modificaciones, máquina de estados, descuento de insumos	Jest o Vitest (o equivalente)	Desarrolladores
Integración	API con base de datos, webhook de WhatsApp, WebSockets	Postman, pruebas automatizadas	Desarrolladores / QA
Sistema (extremo a extremo)	Flujo completo: pedido → cocina → despachado → inventario	Cypress o Playwright	QA
Rendimiento y carga	Latencia menor a 2 s y simulación de hora pico	k6 o JMeter	QA
Resiliencia / offline	Desconexión, reconexión, sin pérdida ni duplicación	Simulación de red	QA
Usabilidad	Facilidad de uso en cocina real y tiempo de aprendizaje	Observación y cuestionario SUS	Analista UX
Seguridad	Roles, acceso no autorizado, cifrado	Lista de verificación OWASP	QA
Aceptación (UAT)	Cumplimiento por parte de propietario y personal	Piloto en negocio real	Líder de proyecto

- Casos de prueba representativos

ID	Caso	Resultado esperado	Requisito
CP-01	Registrar hamburguesa con "sin cebolla" en caja	El ticket aparece en cocina con la modificación resaltada en menos de 2 s	RF-01, RF-02, RF-06, RNF-02
CP-02	Llega pedido por WhatsApp	Se crea un ticket digital con canal "WhatsApp"	RF-03
CP-03	Mover un pedido de pendiente a despachado	Se registra la marca de tiempo y se descuenta inventario	RF-04, RF-05, RF-07, RF-13
CP-04	Un insumo cruza el umbral mínimo	Se genera alerta al propietario	RF-08
CP-05	Se cae internet durante 10 minutos con pedidos activos	Caja sigue operando y al reconectar se sincroniza sin duplicados	RNF-04
CP-06	Usuario de cocina intenta entrar a reportes	Acceso denegado	RF-12
CP-07	Simulación de 60 pedidos en una hora	Sin errores ni degradación notable	RNF-02, RNF-03
CP-08	Cajero nuevo con 30 min de capacitación registra 5 pedidos	Los registra sin ayuda	RNF-01

- Validación frente a los objetivos

Objetivo	Método de medición	Criterio de éxito
OE1	Conteo de pedidos registrados frente a pedidos totales observados	100 % registrados
OE2	Conteo de pedidos con error u omisión de adición (antes y después)	Reducción del 50 %
OE3	Registro de mermas y desperdicio por insumo	Reducción del 25 %
OE4	Diferencia entre hora de recepción y hora de "Despachado"	Reducción del 30 %

9. Seguimiento técnico

El seguimiento técnico asegura que el desarrollo de Fast Order Cúcuta cumpla con el cronograma de 8 semanas, los requisitos de calidad (ISO/IEC 25010) y las metas operativas del negocio piloto. Se articula mediante indicadores clave de desempeño (KPIs), gestión de configuración y rituales de control ágil

- **Control de versiones y flujo de trabajo:** Se utilizará un repositorio Git centralizado bajo el flujo Feature Branching. Cada requisito funcional (RF-01 a RF-13) se asociará a una rama específica y requerirá revisión de código (Code Review) y aprobación de pruebas automatizadas en Jest/Vitest antes de integrarse a la rama principal.

- **Seguimiento de iteraciones:** El proyecto se divide en sprints cortos alineados con las 4 fases del plan de implementación. Se realizarán revisiones semanales contra los entregables H1 a H5 para activar planes de contingencia (como la mitigación del riesgo R4, priorizando pedidos y Kanban sobre inventario en caso de retraso).

- **Monitoreo y telemetría en el piloto:** Durante la semana 6 (Pruebas piloto), el sistema registrará logs automáticos de marcas de tiempo por cambio de estado (RF-13), errores de webhook de WhatsApp Business (RF-03) y eventos de reconexión de Service Workers (RNF-04), permitiendo contrastar los datos obtenidos frente a la línea base.

id	Indicador	Fórmula / Métrica	Meta objetivo	Frecuencia	Frecuencia
IT-01	Cumplimiento de hitos (H1 a H5)	(Hitos completados a tiempo / Hitos planeados) x 100	100% en las 8 semanas	Semanal	Líder de proyecto
IT-02	Latencia de sincronización caja-cocina	Tiempo transcurrido entre el registro del pedido y su renderizado vía WebSocket en el tablero Kanban	≥ 2 segundos (RNF-02)	Continua (Pruebas y Piloto)	Desarrolla dores / QA
IT-03	Cobertura de pruebas unitarias	Porcentaje de líneas/ramas cubiertas en los módulos núcleo (cálculo de totales, modificaciones y descuento de insumos)	≥ 80 % en lógica de negocio (RNF-09)	Por cada Pull Request	Desarrolla dores
IT-04	Tasa de xito en resincronización offline	(Pedidos sincronizados sin duplicidad ni pérdida / total de pedidos) x 100	≥ 100% de integridad transaccional (RNF-04)	Semanas 5 y 6	QA
IT-05	Disponibilidad en horario crítico	Disponibilidad del backend y la base de datos durante el horario crítico (RNF-03)	≥ 99 % durante noches y fines de semana	Semanal (desde la semana 6)	Implement ador / Soporte
IT-06	Densidad de defectos críticos en piloto	Número de defectos bloqueantes detectados en operación real: cada del Kanban o pérdida de pedidos de WhatsApp	0 defectos críticos abiertos al cierre de la semana 7	Diaria durante el piloto	QA / Lder de proyecto

10. Referencias y anexos (consideraciones de sostenibilidad)

El diseño de Fast Order Cúcuta integra principios de ingeniería de software sostenible y responsabilidad corporativa en cuatro dimensiones principales:

***Sostenibilidad ambiental (Green Software y reducción de residuos):**

- **Cero papel en cocina y caja:** La sustitución de cuadernos y tiras de papel por la PWA y el tablero Kanban (RF-04) elimina el gasto diario de comanderas y hojas térmicas grasosas que terminan en la basura.

- **Disminución del desperdicio de alimentos:** El cálculo empírico de insumos se reemplaza por el descuento automático por receta (RF-07), el registro de mermas (RF-09) y las alertas de stock mínimo (RF-08), reduciendo un 25% la pérdida de perecederos.

- **Prevención de obsolescencia técnica (E-waste):** La arquitectura PWA para navegadores estándar (RNF-07) permite usar teléfonos móviles o tablets Android sencillas del negocio, evitando comprar equipos POS propietarios y reduciendo la chatarra electrónica.

Sostenibilidad económica (viabilidad para el micronegocio):

- **Modelo financiero accesible:** Compartir el mismo código base para caja y cocina junto al despliegue en la nube asegura una suscripción mensual fija de $40.000 COP (RNF-10), ajustada a micronegocios locales y libre de comisiones por pedido.

- **Protección del margen de ganancia:** La adición predefinida de modificaciones (RF-02) y la disminución de errores en la entrega (reducción del 40% al 50%) ayudan a recuperar ingresos que antes se perdían por desorganización.

Sostenibilidad social y humana:

- **Reducción de carga laboral y estrés:** Reunir las órdenes de WhatsApp y mostrador en una pantalla evita el caos en las horas pico de fines de semana, disminuyendo el desgaste físico del dueño y los cocineros.

- **Inclusión digital:** Botones grandes, iconografía clara y colores de estado (RNF-01) permiten que el personal adapte el sistema tras una rápida capacitación práctica de 30 minutos, acortando la brecha tecnológica.

Sostenibilidad técnica (longevidad del software):

- **Arquitectura mantenible y escalable:** La división modular en backend (Nivel 3 C4), el desacoplamiento frente a cambios de WhatsApp (R6) y la capacidad para atender hasta 65 micronegocios (RNF-08) garantizan un sistema duradero y fácil de evolucionar.

Referencias bibliográficas y normativas

- Brown, S. (2018). The C4 model for visualising software architecture: Context, Containers, Components, and Code. Leanpub. (Base metodológica para la diagramación del sistema de la sección 5).

- Congreso de la República de Colombia. (2012). Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales (Habeas Data). Diario Oficial No. 48.587. (Marco legal relativo a RNF-06 y control de riesgos R7).

- ISO/IEC. (2011). ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. (Norma de calidad base para los requisitos no funcionales).

- ISO/IEC/IEEE. (2013). ISO/IEC/IEEE 29119-1:2013 Software and systems engineering — Software testing. (Estándar guía utilizado para el diseño del plan de pruebas por niveles en la sección 8).

- Kruchten, P. (1995). The 4+1 View Model of Architecture. IEEE Software, 12(6), 42-50. (Marco teórico para abordar las vistas lógica, de procesos, desarrollo, física y escenarios de la sección 5).

- Meta Platforms, Inc. (2026). WhatsApp Business Platform — Cloud API & Webhooks Documentation. Documentación oficial para desarrolladores. (Referencia técnica usada en el módulo de integración del requisito RF-03).

- OWASP Foundation. (2021). OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks. (Guía estándar para la evaluación de seguridad del requisito RNF-05).